

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I Shaik Ayesha Tabassum(192525074) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

Normalization of University Student-Course Table (Up to 3NF)

Given Unnormalized Table (UNF)

Student_ID	Student_Name	Department	Course_ID	Course_Name	Faculty_Name	Faculty_Phone
S101	Ayesha	CSE	C201	DBMS	Dr. Kumar	9876543210
S101	Ayesha	CSE	C202	Java	Dr. Rani	9876501234
S102	Rahul	ECE	C201	DBMS	Dr. Kumar	9876543210

Issues in the Table

- Student details are repeated for every course.
- Faculty information is repeated for students taking the same course.
- Updating faculty details requires changing multiple rows (Update Anomaly).
- Deleting the last student enrolled in a course may remove course information (Deletion Anomaly).
- Adding a new course without a student is not possible (Insertion Anomaly).

Step 1: First Normal Form (1NF)

Rule: Remove repeating groups and ensure each column contains only one value.

The given table already satisfies 1NF because each field contains atomic values.

Student_ID	Student_Name	Department	Course_ID	Course_Name	Faculty_Name	Faculty_Phone
S101	Ayesha	CSE	C201	DBMS	Dr. Kumar	9876543210
S101	Ayesha	CSE	C202	Java	Dr. Rani	9876501234
S102	Rahul	ECE	C201	DBMS	Dr. Kumar	9876543210

Step 2: Second Normal Form (2NF)

Rule: Remove partial dependencies by separating data based on the whole key.

Assume the primary key is (**Student_ID, Course_ID**).

STUDENT Table

Student_ID (PK)	Student_Name	Department
S101	Ayesha	CSE
S102	Rahul	ECE

COURSE Table

Course_ID (PK)	Course_Name	Faculty_Name	Faculty_Phone
C201	DBMS	Dr. Kumar	9876543210
C202	Java	Dr. Rani	9876501234

ENROLLMENT Table

Student_ID (FK)	Course_ID (FK)
S101	C201
S101	C202
S102	C201

Step 3: Third Normal Form (3NF)

Rule: Remove transitive dependencies.

Here, **Faculty_Phone** depends on **Faculty_Name**, not directly on **Course_ID**. So separate faculty details.

STUDENT Table

Student_ID (PK)	Student_Name	Department
S101	Ayesha	CSE
S102	Rahul	ECE

FACULTY Table

Faculty_ID (PK)	Faculty_Name	Faculty_Phone
F101	Dr. Kumar	9876543210
F102	Dr. Rani	9876501234

COURSE Table

Course_ID (PK)	Course_Name	Faculty_ID (FK)
C201	DBMS	F101
C202	Java	F102

ENROLLMENT Table

Student_ID (FK)	Course_ID (FK)
S101	C201
S101	C202
S102	C201

Final 3NF Database Design

- **STUDENT**(Student_ID, Student_Name, Department)
- **FACULTY**(Faculty_ID, Faculty_Name, Faculty_Phone)
- **COURSE**(Course_ID, Course_Name, Faculty_ID)
- **ENROLLMENT**(Student_ID, Course_ID)

Advantages of 3NF

- Eliminates data redundancy.
- Prevents insertion, update, and deletion anomalies.
- Improves data consistency and integrity.
- Makes database maintenance easier.
- Reduces storage requirements.

Result: The original university table has been successfully normalized from **UNF** → **1NF** → **2NF** → **3NF**, resulting in four well-structured tables: **STUDENT**, **FACULTY**, **COURSE**, and **ENROLLMENT**.

2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

```
mysql> USE company;
Database changed
mysql> CREATE TABLE Employee (
->   Emp_ID INT PRIMARY KEY,
->   Emp_Name VARCHAR(50),
->   Salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Department (
->   Dept_ID VARCHAR(10) PRIMARY KEY,
->   Dept_Name VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Employee_Department (
->   Emp_ID INT,
->   Dept_ID VARCHAR(10),
->   PRIMARY KEY (Emp_ID, Dept_ID),
->   FOREIGN KEY (Emp_ID) REFERENCES Employee(Emp_ID),
->   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> desc Employee_Department;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Emp_ID | int | NO | PRI | NULL | |
| Dept_ID | varchar(10) | NO | PRI | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

```
mysql> INSERT INTO Employee VALUES
-> (101, 'Ayesha', 50000),
-> (102, 'Rahul', 60000),
-> (103, 'Priya', 55000);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> select * from Employee;
+-----+-----+-----+
| Emp_ID | Emp_Name | Salary |
+-----+-----+-----+
| 101 | Ayesha | 50000.00 |
| 102 | Rahul | 60000.00 |
| 103 | Priya | 55000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> INSERT INTO Department VALUES
-> ('D01', 'HR'),
-> ('D02', 'IT'),
-> ('D03', 'Finance');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> select * from Department;
+-----+-----+
| Dept_ID | Dept_Name |
+-----+-----+
| D01 | HR |
| D02 | IT |
| D03 | Finance |
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Employee_Department;
Empty set (0.00 sec)

mysql> INSERT INTO Employee_Department VALUES
-> (101,'D01'),
-> (101,'D02'),
-> (102,'D02'),
-> (103,'D01'),
-> (103,'D03');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Employee_Department;
+-----+-----+
| Emp_ID | Dept_ID |
+-----+-----+
| 101    | D01     |
| 103    | D01     |
| 101    | D02     |
| 102    | D02     |
| 103    | D03     |
+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID,
-> E.Emp_Name,
-> COUNT(ED.Dept_ID) AS Total_Departments
-> FROM Employee E
-> JOIN Employee_Department ED
-> ON E.Emp_ID = ED.Emp_ID
-> GROUP BY E.Emp_ID, E.Emp_Name
-> HAVING COUNT(ED.Dept_ID) > 1;
```

```
+-----+-----+-----+
| Emp_ID | Emp_Name | Total_Departments |
+-----+-----+-----+
| 101    | Ayesha   | 2                 |
| 103    | Priya    | 2                 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT E.Emp_Name,
-> D.Dept_Name
-> FROM Employee E
-> JOIN Employee_Department ED
-> ON E.Emp_ID = ED.Emp_ID
-> JOIN Department D
-> ON ED.Dept_ID = D.Dept_ID
-> WHERE E.Emp_ID IN
-> (
-> SELECT Emp_ID
-> FROM Employee_Department
-> GROUP BY Emp_ID
-> HAVING COUNT(Dept_ID) > 1
-> )
-> ORDER BY E.Emp_Name;
+-----+-----+
| Emp_Name | Dept_Name |
+-----+-----+
| Ayesha   | HR        |
| Ayesha   | IT        |
| Priya    | HR        |
| Priya    | Finance   |
+-----+-----+
4 rows in set (0.00 sec)
```

Justification of the Approach

- The **Employee_Department** table is used because one employee can work in multiple departments.
- The **JOIN** operation combines employee, department, and assignment details.
- **GROUP BY** groups records based on each employee.
- **COUNT(Dept_ID)** counts how many departments each employee belongs to.
- **HAVING COUNT(Dept_ID) > 1** filters only employees assigned to more than one department.
- This approach is efficient, easy to understand, and works well for many-to-many relationships in relational databases.

Result: The queries correctly identify employees working in multiple departments and display both the employee details and the departments they belong to.

•

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Step 1: Consider the Relation

Suppose

R(A, B, C, D)

Functional Dependencies (FDs):

- $A \rightarrow B$
- $B \rightarrow C$
- $A \rightarrow D$

Step 2: Find the Candidate Key

Find the closure of **A**.

A⁺

$A \rightarrow B$

So,

$A^+ = \{A, B\}$

$B \rightarrow C$

$A^+ = \{A, B, C\}$

$A \rightarrow D$

$A^+ = \{A, B, C, D\}$

Since A determines all attributes,

Candidate Key = {A}

Step 3: Check BCNF

BCNF Rule:

For every functional dependency

X $\rightarrow$ **Y**

X must be a **super key**.

Check each FD.

FD1

$A \rightarrow B$

A is a candidate key.

✓ Satisfies BCNF

FD2

$B \rightarrow C$

B is **not** a candidate key.

✗ Violates BCNF

FD3

$A \rightarrow D$

A is a candidate key.

✓ Satisfies BCNF

Since $B \rightarrow C$ violates BCNF,

Relation R is NOT in BCNF.

Step 4: Decompose the Relation

Using the violating FD

$B \rightarrow C$

Create

R1(B, C)

FD:

$B \rightarrow C$

Here B is the key.

Hence R1 is in BCNF.

Remaining relation

R2(A, B, D)

FDs

$A \rightarrow B$

$A \rightarrow D$

A is the key.

Hence R2 is also in BCNF.

Final BCNF Relations

R1

B C

Key = **B**

R2

A B D

Key = **A**

Final Answer

- **Candidate Key:** A
- **BCNF Check:** Not satisfied because **B** $\rightarrow$ **C** violates BCNF.
- **BCNF Decomposition:**
 - **R1(B, C)**
 - **R2(A, B, D)**

Exam Answer (Short Form)

Relation: R(A, B, C, D)

Functional Dependencies:

- $A \rightarrow B$
- $B \rightarrow C$
- $A \rightarrow D$

Candidate Key: A ($A^+ = \{A, B, C, D\}$)

BCNF Check:

- $A \rightarrow B$ ✓
- $B \rightarrow C$ ✗ (B is not a super key)
- $A \rightarrow D$ ✓

BCNF Decomposition:

- R1(B, C)
- R2(A, B, D)

Conclusion: The relation is **not in BCNF** because of the dependency $B \rightarrow C$. After decomposition into **R1(B, C)** and **R2(A, B, D)**, both relations satisfy BCNF.

4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

```
mysql> CREATE TABLE Customer (  
-> Customer_ID INT PRIMARY KEY,  
-> Customer_Name VARCHAR(50) NOT NULL,  
-> Account_No VARCHAR(20) UNIQUE,  
-> Balance DECIMAL(10,2) CHECK (Balance >= 0),  
-> Branch VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.07 sec)  
  
mysql> INSERT INTO Customer VALUES  
-> (101, 'Ayesha', 'ACC1001', 50000, 'Hyderabad'),  
-> (102, 'Rahul', 'ACC1002', 30000, 'Vijayawada'),  
-> (103, 'Priya', 'ACC1003', 70000, 'Hyderabad');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> CREATE VIEW Employee_View AS  
-> SELECT Customer_ID,  
-> Customer_Name,  
-> Account_No,  
-> Branch  
-> FROM Customer;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> SELECT * FROM Employee_View;  
+-----+-----+-----+-----+  
| Customer_ID | Customer_Name | Account_No | Branch |  
+-----+-----+-----+-----+  
| 101 | Ayesha | ACC1001 | Hyderabad |  
| 102 | Rahul | ACC1002 | Vijayawada |  
| 103 | Priya | ACC1003 | Hyderabad |  
+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

```
mysql> CREATE VIEW Manager_View AS  
-> SELECT *  
-> FROM Customer;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> SELECT * FROM Manager_View;  
+-----+-----+-----+-----+-----+  
| Customer_ID | Customer_Name | Account_No | Balance | Branch |  
+-----+-----+-----+-----+-----+  
| 101 | Ayesha | ACC1001 | 50000.00 | Hyderabad |  
| 102 | Rahul | ACC1002 | 30000.00 | Vijayawada |  
| 103 | Priya | ACC1003 | 70000.00 | Hyderabad |  
+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> CREATE VIEW Hyderabad_Customers AS  
-> SELECT *  
-> FROM Customer  
-> WHERE Branch='Hyderabad';  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> SELECT * FROM Hyderabad_Customers;  
+-----+-----+-----+-----+-----+  
| Customer_ID | Customer_Name | Account_No | Balance | Branch |  
+-----+-----+-----+-----+-----+  
| 101 | Ayesha | ACC1001 | 50000.00 | Hyderabad |  
| 103 | Priya | ACC1003 | 70000.00 | Hyderabad |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Justification

- **Views** restrict access to sensitive information based on the user's role.
- **Employee_View** hides the account balance.
- **Manager_View** provides complete customer details.
- **Hyderabad_Customers** displays records for a specific branch only.
- **Constraints** (PRIMARY KEY, NOT NULL, UNIQUE, CHECK) maintain data integrity and prevent invalid or duplicate data.

5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy

Step 1: Unnormalized Relation (UNF)

Suppose the sales database has the following table:

Order_ID	Customer_ID	Customer_Name	Product_ID	Product_Name	Supplier_ID	Supplier_Name
O101	C101	Ayesha	P101	Laptop	S101	Dell
O101	C101	Ayesha	P102	Mouse	S102	Logitech
O102	C102	Rahul	P101	Laptop	S101	Dell

Step 2: Identify the Problems

The table has:

- Repeated customer details.
- Repeated product details.
- Repeated supplier details.
- Update anomaly.
- Insertion anomaly.
- Deletion anomaly.

Step 3: First Normal Form (1NF)

All values are atomic, so the table is already in **1NF**.

Step 4: Second Normal Form (2NF)

Separate partial dependencies.

CUSTOMER Table

Customer_ID (PK)	Customer_Name
C101	Ayesha
C102	Rahul

PRODUCT Table

Product_ID (PK)	Product_Name	Supplier_ID
P101	Laptop	S101
P102	Mouse	S102

SALES Table

Order_ID	Customer_ID	Product_ID
O101	C101	P101
O101	C101	P102
O102	C102	P101

Step 5: Third Normal Form (3NF)

Remove transitive dependency.

Create a separate supplier table.

SUPPLIER Table

Supplier_ID (PK)	Supplier_Name
S101	Dell
S102	Logitech

PRODUCT Table

Product_ID	Product_Name	Supplier_ID
P101	Laptop	S101
P102	Mouse	S102

CUSTOMER Table

Customer_ID	Customer_Name
C101	Ayesha
C102	Rahul

SALES Table

Order_ID	Customer_ID	Product_ID
O101	C101	P101
O101	C101	P102
O102	C102	P101

BCNF Check

Functional Dependencies:

- $\text{Customer_ID} \rightarrow \text{Customer_Name}$
- $\text{Product_ID} \rightarrow \text{Product_Name}, \text{Supplier_ID}$
- $\text{Supplier_ID} \rightarrow \text{Supplier_Name}$

In each relation, the determinant is the **primary key**.

Therefore, **all relations satisfy BCNF**.

Final 3NF/BCNF Relations

1. **CUSTOMER**(Customer_ID, Customer_Name)
2. **SUPPLIER**(Supplier_ID, Supplier_Name)
3. **PRODUCT**(Product_ID, Product_Name, Supplier_ID)
4. **SALES**(Order_ID, Customer_ID, Product_ID)

Conclusion

The original sales table contained redundant data that caused update, insertion, and deletion anomalies. By decomposing it into **CUSTOMER**, **SUPPLIER**, **PRODUCT**, and **SALES** tables, redundancy is eliminated and the schema satisfies **3NF as well as BCNF**, improving data integrity and reducing storage duplication.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

```
mysql> CREATE TABLE Orders (
->   Order_ID INT PRIMARY KEY,
->   Customer_ID INT,
->   Order_Amount DECIMAL(10,2),
->   FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO Customer VALUES
-> (101,'Ayesha','Hyderabad'),
-> (102,'Rahul','Vijayawada'),
-> (103,'Priya','Hyderabad');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Orders VALUES
-> (1,101,1500),
-> (2,101,2500),
-> (3,102,1000),
-> (4,103,3000),
-> (5,103,2000),
-> (6,103,5000);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Customer;
+-----+-----+-----+
| Customer_ID | Customer_Name | City      |
+-----+-----+-----+
| 101         | Ayesha        | Hyderabad |
| 102         | Rahul         | Vijayawada |
| 103         | Priya         | Hyderabad |
+-----+-----+-----+
```

```
mysql> SELECT * FROM Orders;
+-----+-----+-----+
| Order_ID | Customer_ID | Order_Amount |
+-----+-----+-----+
| 1         | 101         | 1500.00      |
| 2         | 101         | 2500.00      |
| 3         | 102         | 1000.00      |
| 4         | 103         | 3000.00      |
| 5         | 103         | 2000.00      |
| 6         | 103         | 5000.00      |
+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> SELECT C.Customer_ID,
->   C.Customer_Name,
->   O.Order_ID,
->   O.Order_Amount
-> FROM Customer C
-> JOIN Orders O
-> ON C.Customer_ID = O.Customer_ID;
+-----+-----+-----+-----+
| Customer_ID | Customer_Name | Order_ID | Order_Amount |
+-----+-----+-----+-----+
| 101         | Ayesha        | 1         | 1500.00      |
| 101         | Ayesha        | 2         | 2500.00      |
| 102         | Rahul         | 3         | 1000.00      |
| 103         | Priya         | 4         | 3000.00      |
| 103         | Priya         | 5         | 2000.00      |
| 103         | Priya         | 6         | 5000.00      |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> SELECT Customer_Name
-> FROM Customer
-> WHERE Customer_ID IN
-> (
->   SELECT Customer_ID
-> FROM Orders
-> GROUP BY Customer_ID
-> HAVING COUNT(Order_ID) > 1
-> );
+-----+
| Customer_Name |
+-----+
| Ayesha        |
| Priya         |
+-----+
2 rows in set (0.00 sec)

mysql> SELECT C.Customer_Name,
->   COUNT(O.Order_ID) AS Total_Orders
-> FROM Customer C
-> JOIN Orders O
-> ON C.Customer_ID = O.Customer_ID
-> GROUP BY C.Customer_Name;
+-----+-----+
| Customer_Name | Total_Orders |
+-----+-----+
| Ayesha        | 2            |
| Rahul         | 1            |
| Priya         | 3            |
+-----+-----+
3 rows in set (0.00 sec)
```


7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

Step 1: Given Relation

Suppose we have the relation:

STUDENT(Student_ID, Hobby, Language)

Sample Data:

Student_ID	Hobby	Language
S101	Cricket	English
S101	Cricket	Hindi
S101	Music	English
S101	Music	Hindi
S102	Reading	Telugu
S102	Reading	English

Step 2: Identify Multi-Valued Dependencies

For each student:

- A student can have **multiple hobbies**.
- A student can know **multiple languages**.

Functional Dependencies (MVDs):

- Student_ID $\twoheadrightarrow$ Hobby
- Student_ID $\twoheadrightarrow$ Language

These are **independent** of each other, causing data redundancy.

Step 3: Problems in the Relation

- Hobby values are repeated for each language.
- Language values are repeated for each hobby.
- Data redundancy increases.
- Insertion, update, and deletion anomalies occur.

Step 4: Check for 4NF

Rule of 4NF:

A relation is in **4NF** if, for every non-trivial multi-valued dependency $X \twoheadrightarrow Y$, X is a super key.

Here:

- $\text{Student_ID} \twoheadrightarrow \text{Hobby}$
- $\text{Student_ID} \twoheadrightarrow \text{Language}$

Since **Student_ID** is not the only key determining both independent sets of values in this combined relation, the relation **violates 4NF**.

Step 5: Decompose into 4NF

STUDENT_HOBBY Table

Student_ID	Hobby
S101	Cricket
S101	Music
S102	Reading

STUDENT_LANGUAGE Table

Student_ID	Language
S101	English
S101	Hindi
S102	Telugu
S102	English

Step 6: Final 4NF Relations

STUDENT_HOBBY

Student_ID	Hobby
S101	Cricket
S101	Music
S102	Reading

STUDENT_LANGUAGE

Student_ID	Language
S101	English
S101	Hindi
S102	Telugu
S102	English

Explanation

- The original relation contained two independent multi-valued attributes (**Hobby** and **Language**).
- Decomposing the relation separates these independent multi-valued dependencies.
- This removes unnecessary repetition and eliminates insertion, update, and deletion anomalies.
- Both decomposed relations satisfy **Fourth Normal Form (4NF)** because each relation contains only one multi-valued dependency.

Conclusion

The original relation violated **4NF** due to the multi-valued dependencies:

- **Student_ID** $\twoheadrightarrow$ **Hobby**
- **Student_ID** $\twoheadrightarrow$ **Language**

After decomposition into:

- **STUDENT_HOBBY(Student_ID, Hobby)**
- **STUDENT_LANGUAGE(Student_ID, Language)**

the schema satisfies **4NF**, eliminating redundancy and improving data integrity.

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

```
mysql> CREATE TABLE Employee (  
->     Emp_ID INT PRIMARY KEY,  
->     Emp_Name VARCHAR(50) NOT NULL,  
->     Email VARCHAR(50) UNIQUE,  
->     Salary DECIMAL(10,2) CHECK (Salary > 0),  
->     Dept_ID INT,  
->     FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql> INSERT INTO Department VALUES  
-> (1,'HR'),  
-> (2,'IT'),  
-> (3,'Finance');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO Employee VALUES  
-> (101,'Ayesha','ayesha@gmail.com',50000,2),  
-> (102,'Rahul','rahul@gmail.com',60000,1),  
-> (103,'Priya','priya@gmail.com',55000,3);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM Employee;  
+-----+-----+-----+-----+-----+  
| Emp_ID | Emp_Name | Email           | Salary | Dept_ID |  
+-----+-----+-----+-----+-----+  
| 101 | Ayesha | ayesha@gmail.com | 50000.00 | 2 |  
| 102 | Rahul | rahul@gmail.com | 60000.00 | 1 |  
| 103 | Priya | priya@gmail.com | 55000.00 | 3 |
```

Integrity Constraints Used

1. PRIMARY KEY

Emp_ID INT PRIMARY KEY

- Ensures every employee has a unique ID.
- Prevents duplicate and NULL employee IDs.

2. NOT NULL

Emp_Name VARCHAR(50) NOT NULL

- Ensures the employee name cannot be empty.
-

3. UNIQUE

Email VARCHAR(50) UNIQUE

- Prevents duplicate email addresses.

4. CHECK

CHECK (Salary > 0)

- Prevents negative or zero salary values.

5. FOREIGN KEY

FOREIGN KEY (Dept_ID)

REFERENCES Department(Dept_ID)

- Ensures that every department ID in the Employee table exists in the Department table.
- Maintains referential integrity.

Conclusion

The original schema lacked proper constraints, resulting in inconsistent data. By applying **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **CHECK**, and **FOREIGN KEY** constraints, the database enforces data integrity, prevents invalid entries, and maintains consistency across related tables. This design is suitable for a reliable and secure database system.

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy

Step 1: Given Relation

Suppose we have the relation:

SUPPLY(Supplier_ID, Product_ID, Project_ID)

Sample Data:

Supplier_ID	Product_ID	Project_ID
S1	P1	J1
S1	P2	J1
S2	P1	J2
S2	P2	J2

Step 2: Identify the Join Dependency

The relation stores:

- Which **supplier** supplies which **product**.
- Which **supplier** works on which **project**.
- Which **product** is used in which **project**.

There is a **join dependency** because the original relation can be reconstructed by joining smaller relations.

Step 3: Problem

The table contains redundant combinations.

This leads to:

- Data redundancy
- Update anomalies

- Insertion anomalies
- Deletion anomalies

Hence, the relation is **not in 5NF**.

Step 4: Decompose into 5NF

SUPPLIER_PRODUCT

Supplier_ID	Product_ID
S1	P1
S1	P2
S2	P1
S2	P2

SUPPLIER_PROJECT

Supplier_ID	Project_ID
S1	J1
S2	J2

PRODUCT_PROJECT

Product_ID	Project_ID
P1	J1
P2	J1
P1	J2
P2	J2

Step 5: Final 5NF Relations

SUPPLIER_PRODUCT

Supplier_ID	Product_ID
S1	P1
S1	P2
S2	P1
S2	P2

SUPPLIER_PROJECT

Supplier_ID	Project_ID
S1	J1
S2	J2

PRODUCT_PROJECT

Product_ID	Project_ID
P1	J1
P2	J1
P1	J2
P2	J2

Step 6: Verification

Joining the three decomposed relations reconstructs the original **SUPPLY** relation without introducing spurious tuples.

Therefore, the decomposition is **lossless** and removes redundancy.

Conclusion

The original relation contained a **join dependency**, causing redundant data. By decomposing it into:

- **SUPPLIER_PRODUCT(Supplier_ID, Product_ID)**
- **SUPPLIER_PROJECT(Supplier_ID, Project_ID)**
- **PRODUCT_PROJECT(Product_ID, Project_ID)**

the schema satisfies **Fifth Normal Form (5NF)**. This decomposition eliminates redundancy, avoids insertion, update, and deletion anomalies, and ensures that the original relation can be reconstructed through a lossless join.

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

```
mysql> select * from customer;
+-----+-----+-----+
| Customer_ID | Customer_Name | City |
+-----+-----+-----+
| 101 | Ayesha | Hyderabad |
| 102 | Rahul | Vijayawada |
| 103 | Priya | Hyderabad |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> ^C
mysql> SELECT DISTINCT C.Customer_Name
-> FROM Customer C
-> INNER JOIN Orders O
-> ON C.Customer_ID = O.Customer_ID;
+-----+
| Customer_Name |
+-----+
| Ayesha |
| Rahul |
| Priya |
+-----+
3 rows in set (0.00 sec)

mysql> CREATE INDEX idx_customer
-> ON Orders(Customer_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT C.Customer_Name,
-> COUNT(O.Order_ID) AS Total_Orders
-> FROM Customer C
-> JOIN Orders O
-> ON C.Customer_ID = O.Customer_ID
-> GROUP BY C.Customer_ID, C.Customer_Name
-> HAVING COUNT(O.Order_ID) > 1;
+-----+-----+
| Customer_Name | Total_Orders |
+-----+-----+
| Ayesha | 2 |
| Priya | 3 |
+-----+-----+
2 rows in set (0.00 sec)
```

Justification

- **JOIN** combines related tables efficiently.
- **Indexing** on Customer_ID speeds up joins and searches.
- **GROUP BY** groups records by customer.
- **HAVING** filters customers based on aggregate results.
- Replacing unnecessary subqueries with joins improves performance and readability.

Conclusion

The inefficient query was optimized by:

- Replacing subqueries with **JOINS**.
- Creating an **index** on the frequently searched column.
- Using **GROUP BY** and **HAVING** for efficient aggregation.

These optimizations improve SQL query performance, reduce execution time, and make the queries easier to maintain.